

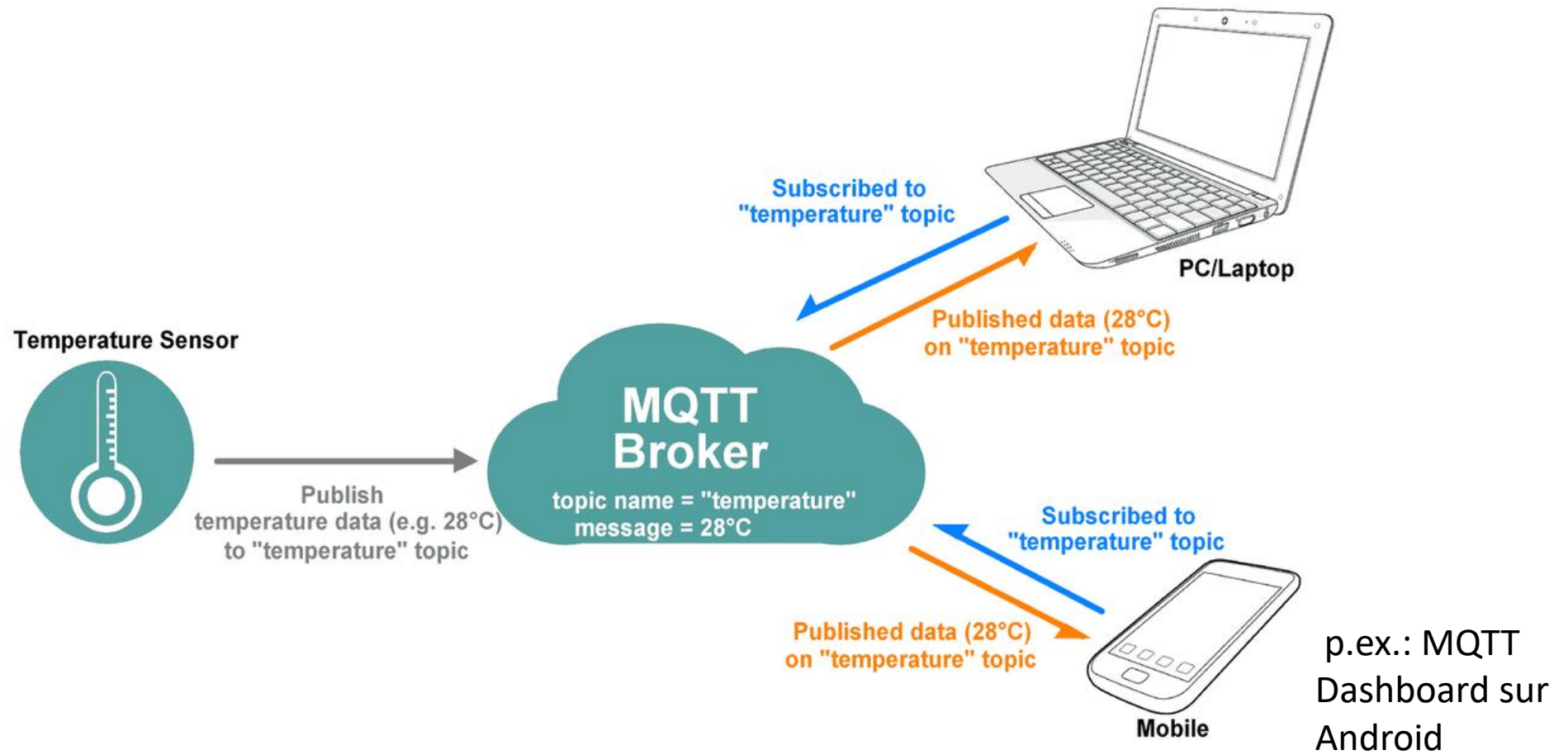


Le protocole MQTT

De Dijcker Sebastien

Haute Ecole de la Province de Liège – Catégorie Technique

1. Introduction



1.1 Définition

- Message Queuing Telemetry Transport
- Protocole de transport de type Client/Serveur
- Pattern d'échange de message de type subscribe/publish:
 - Le client publie des message sur un topic vers le serveur,
 - Le client s'abonne à des topics, et reçoit les messages du serveur concernant ce topic.
- Utilisé pour l'IoT (Internet of Things) et le M2M (Machine to Machine)
- Standard OASIS (Organization for the advancement of Structured Information Standards)

1.2 Fonctionnalités principales

- Nécessite peu de code et est facile à implémenter
- Utilise le protocole TCP/IP
- Quantité de données transmises réduites au maximum
 - **Idéal quand il y a peu de ressources mémoires, une bande passante ou un débit limité**
- Mécanisme de QoS (Quality of Service)
- Possibilité de stocker les messages publiés au niveau du broker
- Sécurité: possibilité d'authentification par username/password

1.3 Applications

- Domotique
- Automobile
- Automation industrielle
- Soins de santé
- Applications mobiles (messagerie, monitoring)
- IoT en général

2. Le protocole

2.1 Control packet

- Pour fonctionner, MQTT échange une série de paquets de contrôle chacun composés de maximum trois parties:

Fixed Header – Présent dans <u>tous</u> les paquets de contrôle
Variable Header – Présent dans certains paquets de contrôle
Payload – Présentes dans certains paquets de contrôle

2.1.1 Control Packet – Fixed Header

Bit	7	6	5	4	3	2	1	0
Byte 1	Type				Flags (type dependant)			
Byte 2	Remaining length							

- Type :
 - 1 – CONNECT : Demande de connexion
 - 2 – CONNACK : ACK de connexion
 - 3 – PUBLISH : Publier un message
 - 4 – PUBACK : ACK de message (QoS1)
 - 5 – PUBREC : Publication reçue (QoS2)
 - 6 – PUBREL : Publication publiée (QoS2)
 - 7 – PUBCOMP : Publication complète (QoS2)

2.1.1 Control Packet – Fixed Header

Bit	7	6	5	4	3	2	1	0
Byte 1	Type				Flags (type dependant)			
Byte 2	Remaining length							

- Type :
 - 8 – SUBSCRIBE : S'abonner à un topic
 - 9 – SUBACK : ACK d'abonnement
 - 10 – UNSUBSCRIBE : Se désabonner d'un topic
 - 11 – UNSUBACK : ACK de désabonnement
 - 12 – PINGREQ : Requête PING
 - 13 – PINGRESP : Réponse PING
 - 14 – DISCONNECT : Notification de déconnexion
 - 15 – AUTH : Echange d'authentification

3. Les Topics

3. Topics

- Les topics sont des identificateurs textuels permettant de différencier la source ainsi que la nature des données
- Les topics peuvent être choisis arbitrairement et sont dépendants de l'application
- On peut créer une arborescence de topics grâce au symbole '/'
 - Exemples: *home/1stFloor/temperature/bedroom*
home/1stFloor/temperature/bathroom
home/gndFloor/temperature/bedroom
- Attention, Les topics sont sensibles à la casse

3.1 Les Wildcards

- Il existe deux Wildcards
 - La dièse (#) peut représenter plusieurs niveaux dans la hiérarchie de topics(topic parent ainsi que tous les enfants). Il doit toujours être le dernier caractère.
 - Exemple: Si un client s'abonne à *'home/1stFloor/temperature/#'*, il recevra les messages publiés sur les topics suivants:
 - *'home/1stFloor/temperature'*
 - *'home/1stFloor/temperature/bedroom'*
 - *'home/1stFloor/temperature/bathroom'*

4.1 Les Wildcards

- Il existe deux Wildcards
 - Le plus (+) peut représenter un seul niveaux dans la hiérarchie de topics.
 - Exemple: Si un client s'abonne à *'home/+/temperature/bedroom'*, il recevra les messages publiés sur les topics suivants:
 - *'home/1stFloor/temperature/bedroom'*
 - *'home/gndFloor/temperature/bedroom'*
 - Exemple 2 : Si un client s'abonne à *'home/+'*, il recevra les messages publiés sur les topics suivants:
 - *'home/1stFloor'*
 - *'home/gndFloor'*



4. Mosquitto

An open source MQTT Broker

4. Mosquitto

- Mosquitto est un logiciel open source permettant de déployer simplement un broker MQTT ainsi qu'un client MQTT
- Disponible pour Windows, Linux, Debian, **Raspberry**, ...

4.1 Mosquitto sur Raspberry

- Sur Raspberry, Mosquitto est disponible dans les repository principal. Dès lors, pour l'installer il suffit de lancer les commandes suivantes:

```
seb@retropie:~ $ sudo apt-get update  
seb@retropie:~ $ sudo apt-get upgrade  
seb@retropie:~ $ sudo apt-get install mosquitto  
seb@retropie:~ $ sudo apt-get install mosquitto-clients
```

- Mosquitto se lance au démarrage du Raspberry.
- Le fichier de configuration se trouve dans `/etc/mosquitto/`
- Un exemple de fichier de configuration complet se trouve dans `/usr/share/doc/mosquitto/examples` (gunzip `mosquitto.conf.gz`)

4.1 Mosquitto sur Raspberry

- Afin de tester si Mosquitto fonctionne correctement, il suffit de s'abonner à un topic dans un terminal (mosquitto_sub):

```
seb@retropie:~ $ mosquitto_sub -h localhost -t "home/1stFloor/temperature/bathroom" -p 1885
```

- Options:
 - -h pour spécifier l'hôte (ip, localhost,...)
 - -t pour spécifier le topic auquel on veut s'abonner (utilisez votre groupe dans le topic)
 - -p pour spécifier le port (1883 par défaut, 1885 sur la carte Nyala)

4.1 Mosquitto sur Raspberry

- Et d'envoyer un message sur ce même topic (mosquitto_pub)

```
seb@retropie:~ $ mosquitto_pub -h localhost -t "home/1stFloor/temperature/bathroom" -p 1885 -m "message de test"
```

- Options:
 - -h pour spécifier l'hôte (ip, localhost,...)
 - -t pour spécifier le topic auquel on veut s'abonner
 - -p pour spécifier le port (1883 par défaut, 1885 sur la carte Nyala)
 - -m pour spécifier le message
 - -u pour le nom d'utilisateur (pas nécessaire)
 - -P pour son mot de passe (pas nécessaire)

5. Envoyer des messages depuis Python

- Il existe une librairie facilitant l'envoi/réception de messages vers un broker MQTT: la librairie PAHO
- Nous voyons une manière simple de l'utiliser dans un script Python:

```
import paho.mqtt.client
#création de l'objet client
monClientMqtt = paho.mqtt.client.Client()
#connexion au serveur (adresse ip, port)
monClientMqtt.connect("127.0.0.1",1885)
#envoi d'un message vers le serveur (topic, message)
monClientMqtt.publish("testTopic","HelloWorld")
```

6. Exercice

- Tester le broker mosquitto avec `mosquitto_pub` et `mosquitto_sub` en ligne de commande
- Publier un message sur le broker depuis python, le réceptionner avec `mosquitto_sub`
- Envoyer la température du BME280 en boucle, toutes les 5 secondes, réceptionner avec `mosquitto_sub`
- Consigne générale: utiliser votre groupe (B11...B14) dans TOUS VOS TOPICS

7. Recevoir des messages MQTT depuis Python

- Après avoir envoyé des messages sur le broker MQTT, il serait idéal d'également pouvoir recevoir des messages depuis ce broker sur un certain topic.
- La librairie Paho offre plusieurs possibilités

7. Recevoir des messages MQTT depuis Python

- Le cas simple: on désire uniquement recevoir des messages.
- Il va falloir faire plusieurs choses:
 - Souscrire au(x) topic(s) sur lesquels nous voulons recevoir des messages
 - Définir une fonction qui sera appelée lors de la réception du message
 - Faire le lien entre cette fonction qu'on a définie et la réception du message
 - Attendre un message indéfiniment

7. Recevoir des messages MQTT depuis Python

- Le cas simple: on désire uniquement recevoir des messages.
- Il va falloir faire plusieurs choses:
 - Souscrire au(x) topic(s) sur lesquels nous voulons recevoir des messages

- Définir une fonction qui sera appelée lors de la réception du message

- Faire grâce à la fonction subscribe:

- Attendre

Une fois la connexion avec le broker établie, nous pouvons souscrire à un topic

```
monClientMqtt.subscribe("topicAuquelOnSAbonne")
```


7. Recevoir des messages MQTT depuis Python

- Le cas simple: on désire uniquement recevoir des messages.
- Il va falloir faire plusieurs choses:
 - Souscrire au(x) topic(s) sur lesquels nous voulons recevoir des messages
 - Définir une fonction qui sera appelée lors de la réception du message
 - Faire le lien entre cette fonction qu'on a définie et la réception du message
 - Att

Ce type de fonction est appelé une fonction de callback, voici comment elle doit être définie:

```
def nomDeMaFonctionDeCallback (client, userdata, msg):  
    #code de la fonction, par exemple  
    print(msg.payload)
```

7. Recevoir des messages MQTT depuis Python

- Le cas simple: on désire uniquement recevoir des messages.
- Il va falloir faire plusieurs choses:
 - Souscrire au(x) topic(s) sur lesquels nous voulons recevoir des messages

- Définir une fonction qui sera appelée lors de la réception d'un message

- Faire le lien entre cette fonction et l'abonnement

- Attacher la fonction à l'abonnement

Ce type de fonction est appelé une fonction de callback, voici comment elle doit être définie:

```
def nomDeMaFonctionDeCallback (client, userdata, msg):  
    #code de la fonction, par exemple  
    print(msg.payload)
```

msg est un objet de type `MQTTMessage`, et contient les informations suivantes:

- topic
- payload
- qos
- retain
- mid (message id)
- properties

7. Recevoir des messages MQTT depuis Python

- Le cas simple: on désire uniquement recevoir des messages.
- Il va falloir faire plusieurs choses:
 - Souscrire au(x) topic(s) sur lesquels nous voulons recevoir des messages
 - Définir une fonction qui sera appelée lors de la réception du message
 - Faire le lien entre cette fonction qu'on a définie et la réception du message
 - Attacher la fonction à la connexion

Il faut préciser à notre connexion quelle fonction appeler lors de la réception d'un message:

```
monClientMqtt.on_message = nomDeMaFonctionDeCallback
```

7. Recevoir des messages MQTT depuis Python

- Le cas simple: on désire uniquement recevoir des messages.
- Il va falloir faire plusieurs choses:
 - Souscrire au(x) topic(s) sur lesquels nous voulons recevoir des messages
 - Définir une fonction qui sera appelée lors de la réception du message
 - Faire le lien entre cette fonction qu'on a définie et la réception du message
 - Attendre un message indéfiniment

Une fois que tout est correctement configuré, nous pouvons entrer dans une boucle infinie en attente de messages grâce à la fonction:

```
monClientMqtt.loop_forever()
```

7. Recevoir des messages MQTT depuis Python

- Le cas simple: on désire uniquement recevoir des messages.
- Il va falloir faire plusieurs choses:
 - Souscrire au(x) topic(s) sur lesquels nous voulons recevoir des messages
 - Définir une fonction qui sera appelée lors de la réception du message
 - Faire le lien entre cette fonction qu'on a définie et la réception du message
 - Attendre un message indéfiniment

Une fois que tout est correctement configuré, nous pouvons entrer dans une boucle infinie en attente de messages grâce à la fonction:

```
monClientMqtt.loop_forever()
```

Mais quel est le problème dans ce cas?

7.1. Recevoir des messages MQTT depuis Python, mais aussi en envoyer

- Afin de régler le problème rencontré au slide précédent (on est dans une boucle infinie, donc impossible d'envoyer des messages), nous allons voir une autre fonction "loop" de la librairie Paho, également appelées sur l'objet Client (comme loop_forever)
- Il s'agit en réalité de deux fonctions:
 - `loop_start()` : qui permet de démarrer une boucle d'attente, mais dans un "thread" séparé, en tâche de fond
 - `loop_stop()` : qui met fin à la tâche de fond lancé au préalable
- Ainsi, entre l'appel à `loop_start` et `loop_stop`, nous pouvons mettre la logique de notre programme, durant laquelle nous pouvons envoyer des messages!

7.2. Parenthèse sur les variables

- A votre avis, que va afficher le code suivant:

```
maVariable = 5

def fctTest():
    maVariable = 6
    print(f"Dans la fonction: {maVariable}")

print(f"Avant appel: {maVariable}")
fctTest()
print(f"Après appel: {maVariable}")
```

7.2. Parenthèse sur les variables

- A votre avis, que va afficher le code suivant:

```
maVariable = 5

def fctTest():
    maVariable = 6
    print(f"Dans la fonction: {maVariable}")

print(f"Avant appel: {maVariable}")
fctTest()
print(f"Après appel: {maVariable}")
```

Résultat de l'exécution:

Avant appel: 5

Dans la fonction: 6

Après appel: 5

La variable "maVariable" dans la fonction n'existe que dans le corps de cette fonction, et n'a aucun lien avec la variable du programme principal!

Mais dès lors, comment pourrait-on modifier quoi que ce soit depuis une fonction de Callback? ➔ le mot clé **global**

7.2. Parenthèse sur les variables

- Dans une fonction, il est possible de préciser qu'une variable est globale à tout le script, et que ce que l'on modifie n'est pas une variable locale à la fonction, mais bien une variable globale. Le code devient alors:

```
maVariable = 5

def fctTest():
    global maVariable
    maVariable = 6
    print(f"Dans la fonction: {maVariable}")

print(f"Avant appel: {maVariable}")
fctTest()
print(f"Après appel: {maVariable}")
```

7.3. Une callback par topic

- Dans la fonction appelée lors de la réception d'un message, nous pourrions distinguer les différents topics des différents messages à l'aide d'alternatives.
- Il existe cependant un méthode plus appropriée: assigner une fonction pour chaque topic:
- La fonction "on_message" sera appelée si le message reçu ne rentre dans aucune autre fonction

7.3. Une callback par topic: Exemple

```
import paho.mqtt.client

def callbackTopicTempChambre(client, userdata, msg):
    print(f"La temperature de la chambre est de {msg.payload}")

def callbackTopicTempBureau(client, userdata, msg):
    print(f"La temperature du bureau est de {msg.payload}")

def callbackParDefaut(client, userdata, msg):
    print(f"Message reçu sur le topic {msg.topic}: {msg.payload}")

monClientMqtt = paho.mqtt.client.Client()
monClientMqtt.connect("192.168.1.130", 1883)
monClientMqtt.subscribe("temp/#")

monClientMqtt.message_callback_add("temp/chambre", callbackTopicTempChambre)
monClientMqtt.message_callback_add("temp/bureau", callbackTopicTempBureau)
monClientMqtt.on_message = callbackParDefaut
monClientMqtt.loop_forever()
```

8. MQTT sur ESP32

- Librairie PubSubClient (Nick O'Leary)
- Voir exemples dans l'IDE Arduino

Sources

- Protocole MQTT: OASIS – MQTT Version 5.0 – 07 March 2019: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.pdf>
- Wiki MQTT: <https://github.com/mqtt/mqtt.github.io/wiki>
- Mosquitto: <https://mosquitto.org/>